Parallel and Distributed Systems

Chapter 6: Synchronization Mechanisms

Prof. Dr. Martin Sträßer

Copyright & Acknowledgments

- All materials (incl. lecture and exercise slides, recordings, exercise tasks and sheets, programming tasks) are provided for personal use only. Reproducing, copying, uploading, sharing, or providing materials to third parties is not permitted

Recap: Thread Components

- Notably, threads within the same process share large parts of the process memory
- However, threads have:
  - Their own stack (for function calls and local variables)
  - Own signal handling (they are signal disposition, but signals are received by individual threads)
  - Own kernel entries (they share process information, but have also individual data)
  - Their own program counter and register states

Process Component	Shared by Threads
Program code	Yes
Stack	No
Heap	Yes
Environment variables	Yes
Open file descriptors	Yes
Signal handlers	Partially
Current working directory	Yes
Kernel metadata	Partially

This makes creating threads lightweight! There is not many data that has to be copied!

Recap: Challenges with Threads

- Read and write access to shared resources can lead to unpredictable behavior

```python
import threading
import time

# Shared resource
counter = 0

def increment_worker(name, num_increments):
    global counter
    for _ in range(num_increments):
        current_value = counter
        # Simulate work to force context switch
        time.sleep(0.000001)
        counter = current_value + 1
    print(f'No. {name} finished')
```

```python
if __name__ == '__main__':
    num_threads = 20
    n = 100
    threads = []
    for i in range(num_threads):
        t = threading.Thread(target=increment_worker, args=(f'{i+1}', n))
        threads.append(t)
        t.start()
    for t in threads:
        t.join()
    print(f'Expected counter: {num_threads * n}')
    print(f'Actual counter:   {counter}')
```

Overview

- Critical Sections, Race Conditions, Atomic Operations
- Locks
- Reentrant Locks
- Condition Variables
- Barriers
- Semaphores
- Events
- Queues for Thread Communication
- Lock-Free Data Structures
- Process Safety and Thread Safety

Critical Sections, Race Conditions, Atomic Operations

Critical Section

- Part of program code where shared resources (memory) is accessed reading **and** writing
- If multiple concurrent accesses to the shared resource happen in this critical section, the results are unpredictable
- Mutual exclusion = only one thread/process can be inside the critical section at the same time

```python
import threading
import time

# Shared resource
counter = 0

def increment_worker(name, num_increments):  1 usage
    global counter
    for _ in range(num_increments):
        current_value = counter
        # Simulate work to force context switch
        time.sleep(0.000001)
        counter = current_value + 1
    print(f'No. {name} finished')
```

Race Conditions

- Without mutual exclusion, results depend on scheduling decisions of the operating system
- This situation is a **race condition**
- Race conditions
  - Cause different symptoms in different executions of the same program code
  - Are very hard to debug

```python
import threading
import time

# Shared resource
counter = 0

def increment_worker(name, num_increments):  1 usage
    global counter
    for _ in range(num_increments):
        current_value = counter
        # Simulate work to force context switch
        time.sleep(0.000001)
        counter = current_value + 1
    print(f'No. {name} finished')
```

1
2
3

- Example
  - Two threads A and B
  - Execution order: A1 – A2 – A3 – B1 – B2 – B3, Result: counter = 2
  - Execution order: A1 – B1 – A2 – B2 – A3 – B3, Result: counter = 1

Atomic Operations

- A critical role in concurrent or parallel programming is played by **atomic operations**
- Atomic operations are indivisible
- They either complete or fail/do not happen
- Other threads/processes can not see intermediate states in the operation
- Core problem in modern operating systems: context switches happen very often (e.g., caused by high-priority processes or time slicing)
- Consequence: Atomicity can only be achieved for very small operations (few CPU instructions, low runtime)

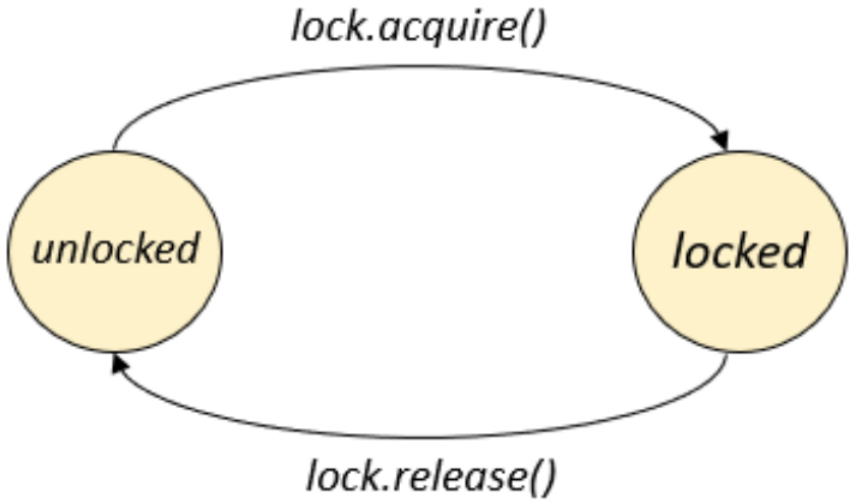
Challenge

- Critical sections can involve quite costly tasks (e.g., reading and writing to a file)
- If they cannot be executed atomically, how to ensure mutual exclusion?
- Conceptual solution: Block concurrent threads/processes (make them not runnable)
- Any other context switch happening in a critical section will have no effect on shared resource

Locks

Locks

- Simplest synchronization mechanism
- When entering a critical section, lock has to be acquired
- When leaving a critical section, lock is released
- Lock can be acquired by one thread at a time
- Acquire and release operations are atomic
- Native support by operating system



Example: Locks

```python
import threading
import time

counter = 0
counter2 = 0
lock = threading.Lock()

def increment_sync():  1 usage
    global counter
    for _ in range(1000):
        lock.acquire()
        temp = counter
        time.sleep(0.000001)
        counter = temp + 1
        lock.release()

def increment():  1 usage
    global counter2
    for _ in range(1000):
        temp = counter2
        time.sleep(0.000001)
        counter2 = temp + 1
```

```python
if __name__ == '__main__':
    threads = [threading.Thread(target=increment_sync) for _ in range(4)]
    start = time.perf_counter()

    for t in threads:
        t.start()
    for t in threads:
        t.join()

    end = time.perf_counter()
    print(counter)
    print(f"Elapsed: {end - start:.6f} seconds")

    threads = [threading.Thread(target=increment) for _ in range(4)]

    start = time.perf_counter()
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    print(counter2)
    end = time.perf_counter()
    print(f"Elapsed: {end - start:.6f} seconds")
```

```
4000
Elapsed: 1.059134 seconds
1005
Elapsed: 0.534999 seconds
```

See code no. 1